Understanding ACT: CVAE and Policy Forward Pass

Explaining the Style Variable Generation

1 Introduction

During the training phase of Action Chunking with Transformers (ACT), the model behaves as a Conditional Variational Autoencoder (CVAE). This means before the main Transformer (the Decoder) can process the vision and proprioception context, we must compute a "style variable," denoted as z. This variable is designed to capture the stochasticity, multi-modality, and specific human nuances of a given demonstration.

This document details the pipeline of the CVAE Encoder up to the point where all arguments are prepared for the main loss function.

2 The CVAE Encoder Inputs

To calculate z during training, the CVAE Encoder is fed a combination of the current state and the actual future actions from the dataset.

In paper terminology, the full observation o_t includes both images and state. The encoder uses $\bar{o}_t$, which is defined as the **observation without the images**. In practice, this means the encoder only sees the low-dimensional physical data. To save on heavy computation (avoiding ResNet passes for the encoder), the images are discarded here.

- **Proprioception ($\bar{o}_t$):** The normalized joint positions of the single-arm state. This is the only part of the observation used by the encoder.
- **Future Actions ($a_{t:t+k}$):** The sequence of normalized k future target joint positions that the human expert actually executed.
- **[CLS] Token:** As described in Section IV.C (page 6) of the original paper, a learnable "Class" token is prepended to the input sequence. Because Transformers output a sequence of the same length as their input, we need a mechanism to condense a list of k actions into a *single* vector. The `[CLS]` token acts as a **Designated Style Extractor**—a dummy variable whose sole purpose is to "listen" to all the other tokens and serve as the final summary bucket.

3 The Style Encoder Forward Pass (Encoder-Only)

The inputs are mapped to the hidden dimension size (e.g., 512) and passed through the Style Transformer. It is crucial to note that this module is an **Encoder-Only** architecture (like BERT)—it does *not* have a decoder. It simply takes the sequence of inputs and outputs an enriched sequence of the exact same length.

This architecture acts as a **Universal Translator**: it reads a "paragraph" of complex human physical movements bidirectionally (all at once instead of sequentially) and translates it into a single "word" (the style vector z).

3.1 Aggregating Information

Inside this Encoder, multi-head self-attention allows the `[CLS]` token to interact with the entire mathematical sequence of future actions and the proprioception vector. By the time the data passes through all the layers:

- The model outputs a sequence of exactly 102 tokens.
- **The Extraction**: The 101 output vectors corresponding to the individual actions and proprioception are completely discarded.
- The single 512-dimensional vector located at the `[CLS]` position is kept. Because it spent the encoding process "listening" to the other tokens, it now contains a compressed mathematical summary of the sequence's "style."

3.2 Predicting the Distribution: ϕ and the Gaussian Assumption

Instead of generating a static tensor, ACT assumes the "style" of a trajectory is a random process following a **Multivariate Gaussian Distribution**. To solve this, the summary `[CLS]` feature is passed through two linear layers to predict the statistical parameters of that distribution:

1. **Mean Vector (μ):** The center of the "style" distribution in the 512-dimensional space.
2. **Variance (σ^2):** A vector representing the diagonal variance (the "uncertainty" or "spread") of each style dimension.

A Note on Notation (ϕ): In Algorithm 1 of the original paper, this prediction step is denoted mathematically as sampling z from a distribution $q_\phi(z|a, \bar{o})$. It is important to realize that ϕ does *not* represent the mean or variance values themselves. Instead, ϕ represents the physical **neural network weights** (the parameters) of the CVAE Style Encoder. The function q_ϕ is the neural network computing the μ and σ^2 vectors based on those weights.

During training, the model performs **Dual Learning**: it simultaneously refines its ability to "translate" human motion (updating the Encoder weights ϕ) while also sharpening its ability to "execute" those movements (updating the Policy Transformer weights θ).

4 Sampling and the Reparameterization Trick

Instead of directly passing the mean as z, we sample from the predicted Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$.

To maintain differentiability for backpropagation during training, a concept called the *reparameterization trick* is applied:

$$z = \mu + \sigma \cdot \epsilon$$

Where ϵ is random noise sampled from a standard normal distribution $\mathcal{N}(0, 1)$. This z vector is finally projected to the standard hidden dimension (e.g., 512).

5 The Policy Transformer: Action Generation

Now that z has been sampled, the model moves to its second major component: the Policy Transformer. Structurally, this is a standard Transformer **Encoder-Decoder** architecture responsible for physically commanding the robot.

A Note on VAE Terminology (π_θ): In Algorithm 1 of the paper (Line 4), you will see the instruction to *"Initialize decoder $\pi_\theta(a|o, z)$"*. This causes a massive terminology clash! In General Variational Autoencoder (VAE) literature, the entire half of the network that translates z back

into the real world is generically called the "Decoder." Physically, however, the ACT "Decoder" is actually the entire Policy Transformer module (which itself contains both an encoder and a decoder). The parameter θ represents the neural network weights of this entire Policy module.

5.1 The Policy Encoder: Building Context

The Policy Encoder takes the normalized sensor data and combines it with the style variable to create a unified, highly-informed understanding of the current "scene." The input sequence is formed by concatenating:

1. **Vision Tokens:** The visual features from the cameras.
2. **Proprioception Token:** The token representing the robot's state.
3. **Style Token:** The projected z vector.

Inside the Policy Encoder, self-attention layers effectively lock all these distinct modalities in a room and allow them to "speak" to each other. By exchanging information, the wrist camera token learns what the human's intended style (z) is, while the style token learns exactly where the robot arm currently sits. The output is a deeply unified "memory" of the environment.

5.2 The Policy Decoder: Predicting the Chunk

While the Encoder builds the context, the Decoder acts as the translator. Its job is to read the Encoder's memory and "translate" it into a new "sentence" of future movements.

- **Queries (The Blank Sentence):** Instead of generating tokens sequentially like ChatGPT, the Decoder is handed k fixed positional embeddings all at once. Think of these as 100 empty "slots" asking the Encoder, "Given this complex context, what should the first move be? What about the second?"
- **Cross-Attention:** Through cross-attention, these empty slots query the rich memory from the Policy Encoder, filling themselves with specific mathematical plans. The result is a sentence of *purely mathematical action tokens* (each still 512-dimensional).
- **The Final Layer ("De-tokenization"):** Because robots cannot understand 512-dimensional abstract tokens, the output of the Decoder is passed through a final Multi-Layer Perceptron (usually a simple linear layer). This crushes the high-dimensional tokens back down into the actual physical motor dimensions (like 6 or 7 normalized joint angles) representing $\hat{a}_{t:t+k}$.

6 Preparing for the Loss Function

By running both the CVAE Encoder (ϕ) and the Policy Transformer (θ), we have successfully generated every mathematical argument needed to compute the two halves of the total loss:

- **KL Divergence Arguments:** We have the predicted parameters (μ, σ^2) from the CVAE Encoder. These are used to compute the KL loss, ensuring the style distribution doesn't stray too far from a standard normal prior $\mathcal{N}(0, I)$.
- **Reconstruction Arguments:** We now have the Policy Decoder's final predictions ($\hat{a}_{t:t+k}$). We compare these directly to the ground truth human actions ($a_{t:t+k}$) using an L1 smooth reconstruction loss.

Simple Summary: To recap the forward pass, we take the expert actions and joint state to "translate" them into a style distribution (μ, σ^2). We sample a style vector z from this distribution and feed it into the main Policy Transformer alongside the camera images. This produces a predicted chunk of 100 actions. We now have everything we need to calculate how far off our predictions were (L1 Loss) and how messy our translation was (KL Loss).